

Auteur : Briek Wozniak

Inhoud van de colleges

De lessenreeks startte met een eenvoudige vraag: wat is softwarekwaliteit en hoe meten we dit? Aangezien softwarekwaliteit vaak gezien wordt als "de code werkt", leerden we dat kwaliteit in werkelijkheid veel breder is. We maakten kennis met het ISO 25010-model, een internationale standaard die kwaliteit opdeelt in een aantal hoofdcategorieën, waaronder functionele geschiktheid, prestatie-efficiëntie, beveiliging, betrouwbaarheid en onderhoudbaarheid. Elke categorie bestaat bovendien uit meerdere sub-karakteristieken, wat duidelijk maakte dat kwaliteit geen binair begrip is. Een belangrijk inzicht was dat kwaliteit sterk afhangt van het perspectief van de stakeholder: een developer hecht vooral belang aan onderhoudbaarheid en testbaarheid, terwijl een eindgebruiker voornamelijk kijkt naar usability, performance en betrouwbaarheid.

Vervolgens bespraken we de Software Development Life Cycle (SDLC) en de impact van non-quality. Aan de hand van echte voorbeelden, zoals de mislukte lancering van de Ariane 5-raket en grootschalige IT-storingen op luchthavens, werd duidelijk hoe zwaar de gevolgen van softwarefouten kunnen zijn. Een cruciaal concept hierbij was Shift Left Testing: hoe vroeger in de SDLC een fout werd opgespoord, hoe lager de kost om deze te herstellen. Fouten die pas in productie ontdekt worden, zijn exponentieel duurder en risicovoller. Dit maakte duidelijk dat software kwaliteit een continu proces is en geen laatste stap waar je dan alles even opruimt en netjes schrijft.

In het tweede deel verschoof de focus naar Quality by Design. Kwaliteit ontstaat niet toevallig, maar moet bewust ontworpen worden vanaf de eerste requirements. We zagen dat vage of onvolledige requirements vaak de hoofdoorzaak zijn van falende projecten. Om dit te vermijden, leerden we technieken zoals Event Storming en Design Thinking, die helpen om businessprocessen, domeinkennis en gebruikersbehoeften correct te vertalen naar technische oplossingen. Het analyseren van de volledige User Journey bleek hierbij essentieel om software te bouwen die niet enkel technisch correct is, maar ook echte waarde levert.

Daarna doken we diep in Software Architectuur. Architectuur werd omschreven als "de beslissingen die belangrijk zijn en moeilijk te veranderen". We bespraken ontwerpprincipes die helpen om complexiteit beheersbaar te houden. Separation of Concerns bleek hierbij een kernprincipe: elk onderdeel van het systeem heeft één duidelijke verantwoordelijkheid. Dit principe hangt nauw samen met Loose Coupling en High Cohesion, waarbij componenten intern sterk samenhangen maar zo weinig mogelijk afhankelijk zijn van andere componenten. Ook principes zoals Single Source of Truth en KISS (Keep It Simple, Stupid) kwamen aan bod als richtlijnen om onnodige complexiteit te vermijden.

We bekeken verschillende architectuurparadigma's die deze principes concreet maken, zoals Layered Architecture, Microservices en Domain Driven Design (DDD). Vooral DDD maakte indruk door de nadruk op een Ubiquitous Language, waarbij developers en domeinexperts dezelfde terminologie gebruiken. Dit vermindert miscommunicatie en zorgt ervoor dat de code zelf een weerspiegeling wordt van het businessdomein.

Tot slot van de eerste twee colleges werd de bruggetje gemaakt naar de code via Engineering Practices. Principes zoals SOLID, DRY en GRASP helpen om architecturale keuzes correct door te trekken tot op codeniveau. Het centrale idee hierbij was dat development de architectuur tot leven brengt: zelfs een goed doordacht ontwerp faalt als de implementatie niet modulair, leesbaar en testbaar is.

Het derde hoorcollege bouwde hier logisch op voort en zoomde verder in op testing, automatisatie en deployment als fundamenteën van betrouwbare software. We startten met het principe "meten is weten": kwaliteit kan enkel verbeterd worden als ze objectief gemeten wordt. We leerden verschillende methodes om kwaliteit te meten, variërend van formele methoden (zoals model checking en formele verificatie in veiligheid-kritische systemen) tot structurele metingen zoals cyclomatische complexiteit, code duplicatie en maintainability-index. Tools zoals SonarQube illustreren hoe deze metingen automatisch kunnen worden afgedwongen via quality gates. Deze tools worden in grote bedrijven ook veel gebruikt.

Daarnaast bespraken we dynamische metingen, waarbij software effectief uitgevoerd wordt in testomgevingen. Hierbij kwamen verschillende testtypes aan bod: unit testing, integratietesting, end-to-end testing, performance testing en security testing. De testpiramide werd voorgesteld als leidraad, met een sterke nadruk op unit tests omdat deze snel, goedkoop en stabiel zijn. Tegelijk werd gewaarschuwd voor blind vertrouwen in metrics zoals code coverage: een hoog percentage zegt weinig als de tests geen relevant gedrag testen.

Een belangrijk concept was Test Driven Development (TDD), waarbij tests het gewenste gedrag definiëren voor de implementatie. Tests werden voorgesteld als levende documentatie en refactoring als een kernactiviteit. Dit sluit nauw aan bij het idee van Shift Left Testing, waarbij kwaliteit al tijdens development wordt ingebouwd.

De colleges eindigde met deployment en operations. We vergeleken traditionele releaseprocessen, gebaseerd op grote releases en strikte controle, met DevOps-principes, die inzetten op kleine, frequente releases en doorgedreven automatisatie. CI/CD-pipelines maken het mogelijk om builds, tests en deployments automatisch en reproduceerbaar uit te voeren. In productie verschuift de focus naar monitoring en observability, waarbij metrics, logs en tracing nodig zijn om incidenten snel te detecteren en de root cause te achterhalen. Begrippen zoals SLA's en SLO's verduidelijkten het verschil tussen externe beloftes en interne kwaliteitsdoelstellingen. Als afsluiter werd Chaos Engineering aangehaald als een geavanceerde techniek om de robuustheid van systemen te valideren door bewust fouten te introduceren.

Belangrijkste lessons learned

1. Shift Left Testing is essentieel voor kostenefficiëntie. Het inzicht dat het herstellen van een bug in de requirements-fase of development-fase vele malen goedkoper is dan na de release, was voor mij de belangrijkste les. In eerdere projecten zag ik testen vaak als iets dat op het einde gebeurt. Nu begrijp ik dat testen en kwaliteitsbewaking een continu proces moet zijn dat zo vroeg mogelijk start.
2. ISO 25010 is breder dan enkel functionaliteit. Ik focuste me vroeger vooral op "werkt het" (Functionele geschiktheid). De lessen leerden me dat aspecten zoals Onderhoudbaarheid (hoe makkelijk kan ik het aanpassen?) en Overdraagbaarheid (werkt het op andere omgevingen?) minstens zo belangrijk zijn voor de levensduur van software. Zonder deze focus creëer je direct "legacy code".
3. Architectuur gaat over het beheren van complexiteit en afhankelijkheden. Het idee dat componenten intern sterk samenhangend moeten zijn, maar zo min mogelijk van elkaar afhankelijk, verklaart waarom sommige van mijn eerdere projecten in "spaghetti code" veranderden. Een goede architectuur voorkomt dat een wijziging in module A de hele applicatie doet omvallen.

4. Het belang van de Ubiquitous Language (DDD). Bij Domain Driven Design leerden we over het gebruik van een universele taal die zowel door developers als domeinexperts wordt gebruikt. Miscommunicatie tussen "business" en "IT" is een grote bron van fouten (hiervoor zagen we het voorbeeld met al de versies van een schommel in een boom). Door dezelfde terminologie in de code te gebruiken als in de bedrijfsprocessen, wordt de software leesbaarder en correcter.

Zelfreflectie

Een project waar ik deze lessen direct aan kan relateren, is een opdracht waarbij ik een dashboard moest bouwen voor het visualiseren van IoT-data van een draaiende zonnepaneel, geschreven in Python (Django).

Destijds koos ik voor een monolithische aanpak. Alle logica van sensordata ophalen tot verwerking en HTML-rendering zat verweven in enkele grote bestanden. Met de kennis uit de lessen zie ik nu duidelijk dat ik het Separation of Concerns-principe volledig negeerde. Mijn UI-laag was rechtstreeks gekoppeld aan de data-laag, wat leidde tot Tight Coupling en Low Cohesion. Elke wijziging had onvoorspelbare neveneffecten, wat het systeem fragiel maakte en slordig. Ook als ik iets wou aanpassen moest ik steeds heel lang zoeken naar waar alles stond.

Ook op vlak van testing en deployment maakte ik een aantal fouten. Er waren geen unit tests en geen integratietests. Alles werd manueel getest vlak voor oplevering. Hierdoor ontdekte ik fouten zeer laat (of niet), wat leidde tot stress en haastige fixes. Ook als ik een error wou oplossen was het vaak moeilijk om te vinden wat de oorzaak was door de Tight Coupling. Met de inzichten rond Shift Left Testing en CI/CD begrijp ik nu dat dit een structureel probleem was.

Op het vlak van operations en security schoot ik eveneens tekort. De applicatie draaide met één database-account met volledige rechten, zonder monitoring of logging die inzicht gaf in fouten of performanceproblemen.

Hoe zou ik het nu aanpakken? Ik zou starten met een duidelijke architectuur, opgesplitst in lagen of services, met expliciete contracten tussen componenten. Testing zou integraal deel uitmaken van het ontwerp, met unit tests voor logica en integratietests voor kritische flows. Ik zou vanaf dag één een CI/CD-pipeline opzetten, gekoppeld aan automatische kwaliteitscontroles. In productie zou ik monitoring voorzien zodat problemen zichtbaar worden vóór ze impact hebben op gebruikers. Kortom: ik zou niet enkel software bouwen die werkt, maar software die betrouwbaar en beheersbaar is.

Wat ik voor een webdevelopmentproject voor onze Chiro ook bijzonder interessant vond, was de GitHub-branchingstrategie. Hierbij wordt gewerkt met een development branch, meerdere feature branches, een release branch, een hotfix branch en een master branch. Het was interessant om te zien hoe professionele softwareteams op een gestructureerde en efficiënte manier samenwerken. In het Chiro-project, waaraan ik samen met één andere persoon werk, werd eerst hoofdzakelijk op de main branch gewerkt. Af en toe maakten we een feature branch aan wanneer de implementatie van een feature langer duurde dan verwacht, maar in de praktijk werd bijna alles rechtstreeks op main gecommited. Dit zorgde voor problemen bij het samenwerken: mergen was nauwelijks mogelijk en conflicten waren moeilijk beheersbaar. Vaak kwam het erop neer dat ik mijn wijzigingen moest kopiëren, alles moest revert-en, de code van de andere persoon moest pullen en vervolgens mijn eigen

code opnieuw handmatig moest integreren. Dit was niet alleen amateuristisch, maar ook bijzonder inefficiënt en tijdrovend. Door over te stappen op een werkwijze met release branches, die rechtstreeks gesynchroniseerd worden met onze VPS, kunnen we nu op een veel ordelijkere manier samenwerken. We hebben daarbij niet het volledige branchingmodel overgenomen: onze release- en hotfix-branch zijn bijvoorbeeld samengevoegd tot één enkele branch. Toch zorgt deze aanpak voor aanzienlijk meer structuur, een beter overzicht van wie wanneer welke wijzigingen heeft aangebracht, en een vlottere samenwerking waarbij code gecontroleerd kan worden voor deze gemerged wordt.

Appreciatie van de lessenreeks

Ik heb de lessenreeks als bijzonder waardevol ervaren omdat ze een duidelijke brug slaat tussen academisch programmeren en professionele softwareontwikkeling zoals het dus echt in de praktijk verloopt. De combinatie van kwaliteit, architectuur, testing en operations gaf een duidelijk beeld van wat “goede software” werkelijk betekent. Vooral de nadruk op meetbaarheid, automatisatie en verantwoordelijkheid in productie vond ik zeer nuttig; dit zijn aspecten die in klassieke programmeervakken vaak onderbelicht blijven.

De lessen maakten ook duidelijk dat softwareontwikkeling geen individuele activiteit is, maar een samenwerking tussen development, QA en operations. De DevOps-visie en de focus op feedbackloops hebben mijn kijk op software-engineering fundamenteel veranderd.

Een klein verbeterpunt blijft dat sommige architectuur- en frameworkconcepten abstract aanvoelden. Meer praktische, kleinschalige cases zouden helpen om deze theorie duidelijker te maken. Desondanks heeft deze lessenreeks mijn begrip van softwarekwaliteit aanzienlijk verdiept en vormt ze een stevige basis voor mijn verdere opleiding en toekomstige carrière als informaticus.